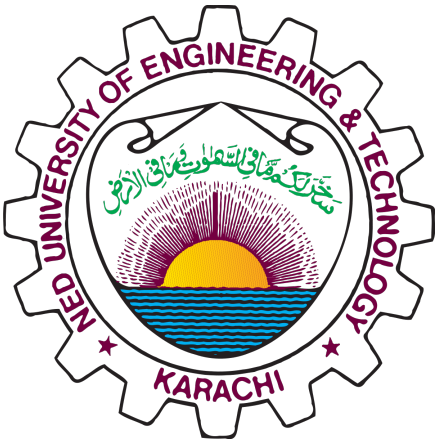


NED University
of Engineering and Technology

Team Reference Document

Team $(ttl)^3$

DP

0-1 Knapsack You are given n items with i_{th} item having weight w_i and value v_i. Given a knapsack with max Weight capacity W. Find the max value you can have.

Note that each item can only be taken **once**.

Time Complexity: $O(nW)$ **Space Complexity:** $O(W)$

```
int knapsack(int n, vector<int>& weights, vector<int>& values, int W) {
    vector<int> dp(W + 1, 0);
    for(int i = 0; i < n; i++) {
        for(int j = W; j >= weights[i]; --j) {
            dp[j] = max(dp[j], values[i] + dp[W - j]);
        }
    }
    return dp[W];
}
```

Complete Knapsack You are given n items with i_{th} item having weight w_i and value v_i. Given a knapsack with max Weight capacity W. Find the max value you can have.

Note that any item can be taken **unlimited times**.

Time Complexity: $O(nW)$ **Space Complexity:** $O(W)$

```
int knapsack(int n, vector<int>& weights, vector<int>& values, int W) {
    vector<int> dp(W + 1, 0);
    for(int i = 0; i < n; i++) {
        for(int j = weights[i]; j <= W; ++j) {
            dp[j] = max(dp[j], values[i] + dp[W - j]);
        }
    }
    return dp[W];
}
```

Multiple Knapsack You are given n items with i_{th} item having weight w_i and value v_i. Given a knapsack with max Weight capacity W. Find the max value you can have.

Note that each item can be taken k_i **times**.

Time Complexity: $O(n \sum \log k_i)$ **Space Complexity:** $O(W)$

```
// Just break each item in powers of 2 and treat each division as single item and apply 01 knapsack
```